



DEMO PLAYBOOK

RunCanon Show and Tell

Complete demo playbook covering admin setup, CLI workflows, the human approval gate, MCP integration, and org-wide skill governance, aligned with the RunCanon Show and Tell Request.

DASHBOARD

<http://127.0.0.1:3000>

DEMO ENGINEER

blessed@runcanon.ai

FORMAT

10-phase walkthrough

RunCanon Show and Tell, Complete Demo Playbook

Aligned with RunCanon_Show_and_Tell_Request.pdf (ordered trajectories, human approval, living skill library, harness-agnostic delivery, org governance).

WHY THIS MATTERS

As more teams adopt AI coding agents, the knowledge of how to do a task well, which tools to call, in what order, with what guardrails, lives in scattered chat transcripts and individual habits instead of a shared, governed asset. Without a deliberate skill management layer, organizations end up with three recurring problems: duplicated effort as each engineer or team rediscovers the same workflow from scratch, inconsistent quality as only the most experienced people produce reliable agent runs, and no audit trail for what an agent was actually instructed to do.

A skill management system turns that tacit knowledge into a versioned, reviewable artifact. That unlocks four concrete benefits for an org: faster onboarding, since new engineers and new agents inherit proven workflows instead of starting cold; consistency across teams and harnesses, since the same vetted skill runs the same way in Claude Code, Cursor, or Copilot; governance and compliance, since every skill has an owner, an approval history, and metrics rather than being an untracked prompt someone pasted once; and continuous improvement, since skills can be updated as workflows drift instead of silently going stale. This is the organizational case for RunCanon, independent of the specific mining technique.

Dashboard: `http://127.0.0.1:3000`

Default admin: `admin@runcanon.ai` / `KeyBoard@2021` (password reset on first login)

Demo engineer: `blessed@runcanon.ai`

Use any project repo on your Mac, `cd` into it before CLI and MCP steps. Commands below use

`--project .`.

INSPIRATION

RunCanon's design is a direct response to a 2026 paper, *Automating SKILL.md Generation for Computer-Using Agents via Interaction Trajectory Mining* (Hao & Li, arxiv.org/pdf/2606.20363). The paper mines raw GUI trajectories into clustered skills and tries to train an autonomous policy on top of them. Its clusters are readable on the source benchmark, but that readability never translates into a working policy.

5 / 8

clusters reach 0.95+ purity against ground-truth labels on the source benchmark

18.5% to 20.5%

GRPO-trained skill-step accuracy gain, barely above zero-shot

beaten

a trivial frequency baseline consistently outperforms the trained policy

RunCanon takes the opposite bet on the same question: mine ordered tool and prompt sequences instead of raw clicks, score by real outcomes instead of a synthetic reward model, keep a human approval gate on by default, and output reviewable skill specs rather than a black-box policy. Every phase in this playbook is built to demonstrate one of those design choices live.

Demo layout

Component	Where it runs
Dashboard + API	http://127.0.0.1:3000 (already running)
CLI	Installed on your Mac (Guide, Install CLI)
Claude Code MCP	Host <code>runcanon mcp</code> (Phase 6)

Install the CLI once from the dashboard Guide (requires Node.js 20+):

```
curl -fsSL http://127.0.0.1:3000/api/releases/install.sh | bash
runcanon --version
```

Phase 1. Admin: platform setup and user creation (8 min)

1.1 Sign in as admin

- 1. Open http://127.0.0.1:3000/login
- 2. Sign in as `admin@runcanon.ai`
- 3. Complete password reset if prompted

1.2 Configure LLM provider (required for mining)

- 1. Admin to Providers
- 2. Enable Anthropic (or OpenAI / Ollama)
- 3. Paste API key, Save
- 4. Test connection until green

Demo line: "Keys live in the encrypted data volume, not in git or YAML."

1.3 Create engineer account (or confirm existing)

Admin to Users to Create user

Field	Example
Email	<code>blessed@runcanon.ai</code>
Name	Blessed Uyo
Role	Engineer
Password	(your demo password)
Require password reset	On (recommended for live demo)

If `blessed@runcanon.ai` already exists, skip creation and use Force password reset only if needed.

1.4 Create a group (optional, for assignment demo)

Admin to Groups to Create

- Name: `Security Team`
- Add `blessed@runcanon.ai` as member

1.5 Admin CLI login

Install CLI from Guide if you have not already, then:

```
runcanon login --server http://127.0.0.1:3000 --email admin@runcanon.ai --password  
'YOUR_ADMIN_PASSWORD'  
runcanon whoami  
runcanon --version  
runcanon --help
```

Pass: `whoami` shows admin and server URL.

Demo line: "CLI auth is the same token MCP uses in connected mode. Stored in `~/.runcanon` on your Mac."

Phase 2. Engineer: login, workspace, harnesses (5 min)

2.1 Browser login

1. Sign out as admin
2. Sign in as `blessed@runcanon.ai`
3. Complete password reset if required
4. Header shows Blessed Uyo's workspace

2.2 Enable harnesses

1. Guide (or Settings)
2. Enable: Claude Code, Cursor, Copilot, Codex
3. Save harnesses

2.3 Engineer CLI login

```
runcanon logout  
runcanon login --server http://127.0.0.1:3000 --email blessed@runcanon.ai --password  
'YOUR_ENGINEER_PASSWORD'  
runcanon whoami
```

Pass:

```
Server: http://127.0.0.1:3000
User:   blessed@runcanon.ai
```

2.4 Optional: browser-based CLI login

```
runcanon logout
runcanon login --server http://127.0.0.1:3000 --browser
runcanon whoami
```

Use when you prefer cookie-based auth instead of passing passwords on the command line.

Phase 3. CLI: full command walkthrough (15 min)

All commands use the host `runcanon` CLI from your project repo (`cd` there first). Use `--project .` for the current directory.

3.1 init, non-destructive project bootstrap

```
cd /path/to/your/project
runcanon init --project .
ls runcanon.config.yaml
ls .runcanon/skills/active 2>/dev/null || true
```

Creates `runcanon.config.yaml` and `.runcanon/` only. Does not modify application code.

3.2 Seed demo trajectories (CVE triage cluster)

Dashboard: Trajectories to Upload, add one or more JSONL session logs.

Or place files under `.runcanon/trajectories/` in your repo:

```
mkdir -p .runcanon/trajectories
# copy or export your agent session JSONL files here
ls .runcanon/trajectories/
```

Demo line: "Three repeated sessions with preserved tool order. That is the paper's missing piece."

3.3 mine, upload JSONL plus remote clustering and LLM proposals

Takes 2 to 5 minutes. Keep the terminal open.

```
runcanon mine --project . \
  --source .runcanon/trajectories
```

Pass: Terminal prints proposal table and link to `/proposals`.

Dashboard check:

1. Trajectories, episodes visible
2. Proposals, pending create/update rows
3. Overview, counts updated

Alternative (dashboard-only re-mine):

1. Settings to Run mining now
2. Wait 2 to 5 minutes

3.4 review, terminal approval (local-only mode note)

In connected mode (after `login`), approval happens in the dashboard, not CLI. `review` applies to offline/local projects:

```
runcanon review --project .
```

Interactive: approve / reject / skip each pending proposal.

Auto-approve (local dev only):

```
runcanon review --project . --auto-approve
```

Show and Tell: Approve in Proposals UI to demonstrate the human gate.

3.5 export, write harness files to your Mac project

```
runcanon export -h claude --project .
runcanon export -h all --project .
ls .claude/skills/*/SKILL.md
ls .cursor/skills/*/SKILL.md 2>/dev/null || true
ls .github/instructions/*.instructions.md 2>/dev/null || true
cat CLAUDE.md 2>/dev/null | head -20
```

Export to server workspace storage only (no local files):

```
runcanon export -h all --project . --workspace
```

3.6 import, Git skill catalog (admin/curator)

Import UC security skills into org library (requires admin or curator login):

```
runcanon logout
runcanon login --server http://127.0.0.1:3000 --email admin@runcanon.ai --password
'YOUR_ADMIN_PASSWORD'

runcanon import \
  --repo https://github.com/your-org/your-skills-repo \
  --branch main \
  --destination org \
  --token "$GITHUB_PAT_IF_PRIVATE"
```

Destination options: `workspace`, `org`, `proposal`. Skip LLM enrichment:

```
runcanon import --repo https://github.com/anthropics/skills --branch main --destination
proposal --no-enrich
```

3.7 logout / whoami

```
runcanon whoami
runcanon logout
runcanon whoami
runcanon login --server http://127.0.0.1:3000 --email blessed@runcanon.ai --password
'YOUR_ENGINEER_PASSWORD'
```

Phase 4. Human approval and skills library (8 min)

Core Show and Tell beat: proposals are suggestions until a human approves.

4.1 Proposals board

1. Proposals, open each pending item
2. Create, new workflow from trajectory cluster
3. Update, refine existing active skill
4. Approve the strongest (e.g. CVE triage or dependency audit)
5. Reject a weak or duplicate proposal

Demo line: "Rejected proposals never reach Skills to Active."

4.2 Skills library

1. Skills, filter Active
2. Open approved skill
3. Show workflow steps, triggers, validation, metrics (frequency, success rate, coherence)
4. Compare to a static SKILL.md in your repo (if you have one)

Demo line: "RunCanon is a living library with metrics, not a one-off markdown file."

4.3 Goal alignment (Overview)

Set goals with the CLI (updates your workspace when logged in, or local `runcanon.config.yaml` offline):

```
runcanon goals set \  
  "Automate CVE triage and vulnerability prioritization" \  
  "Audit dependencies for known security issues"  
  
runcanon goals list  
runcanon goals add "Reduce manual security review toil"
```

Admin: set goals on another workspace with `--workspace WORKSPACE_ID` (from Settings, Your workspace, or Admin, Users).

In Claude Code:

```
Use runcanon_set_goals with goals ["Automate CVE triage and vulnerability prioritization",  
"Audit dependencies for known security issues"].  
Use runcanon_get_goals.
```

1. Open Overview
2. Show the Goal Alignment gauge (0% when no goals are configured)
3. After setting goals and re-mining, refresh and discuss the updated score

Demo line: "We score how well recent trajectories align with stated team goals."

Phase 5. Intelligent updates and relevance (7 min)

Show how RunCanon keeps skills current without silent auto-deploy.

5.1 Record new trajectory activity

Append a session or re-upload JSONL via Trajectories to Upload, or add another file under `.runcanon/trajectories/`:

```
# optional: add another JSONL file locally  
ls .runcanon/trajectories/  
  
runcanon mine --project . --source .runcanon/trajectories
```

Or upload via API (replace `PATH/TO/your-session.jsonl`):


```
TOKEN=$(python3 -c 'import json; print(json.load(open("$HOME/.runcanon/credentials.json"))
["token"]))'
curl -X POST http://127.0.0.1:3000/api/trajectories \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  --data "$(jq -n --rawfile c PATH/T0/your-session.jsonl \
    '{filename:"session-rerun.jsonl", content:$c}')
```

5.2 Update proposals

1. Proposals, new update proposal against existing active skill
2. Show confidence, coherence, goal alignment in proposal reason
3. Approve update or reject if downgrade

Demo line: "Mining proposes updates when workflows drift. Humans still approve."

5.3 Autonomy (opt-in auto-approve)

1. Autonomy, default review / ask
2. Explain auto-approve is opt-in
3. (Optional) raise level, save, discuss trade-offs

Demo line: "Stay reviewable before deployment, matching the paper."

Phase 6. Claude Code MCP setup (5 min)

6.1 Register MCP server (project-scoped)

From your project repo:

```
cd /path/to/your/project
claude mcp remove runcanon 2>/dev/null || true
claude mcp add runcanon -s project -- runcanon mcp
claude mcp list
```

Ensure you are logged in (`runcanon whoami`) before starting Claude Code.

6.2 Approve MCP in Claude Code

```
cd /path/to/your/project
claude
```

On first connect, approve the `runcanon` MCP server when prompted. Type `/mcp` to verify it is connected.

6.3 Verify credentials

MCP reads `~/runcanon/credentials.json` on your Mac. Engineer must be logged in:

```
runcanon whoami
```

Phase 7. All 14 MCP tools in Claude Code (20 min)

Use natural language in Claude Code. Each tool maps to a demo beat. Use the absolute path to your project repo in tool calls that accept `projectPath`.

#	MCP tool	Paste in Claude Code
1	runcanon_list_skills	Use runcanon_list_skills and summarize active, proposed, and org skills.
2	runcanon_get_skill	Use runcanon_get_skill for audit-dependencies-known-cves and summarize the workflow steps.
3	runcanon_get_skill + download	Use runcanon_get_skill for audit-dependencies-known-cves with writeLocally true and harnesses ["claude"]. Use projectPath with the absolute path to this repo.
4	runcanon_list_proposals	Use runcanon_list_proposals and list pending vs applied proposals with recommendations.
5	runcanon_approve_proposal	Try runcanon_approve_proposal for a pending proposal. (Expect blocked in connected mode; explain dashboard is source of truth.)
6	runcanon_reject_proposal	Try runcanon_reject_proposal for a pending proposal. (Same connected-mode block.)
7	runcanon_mine	Use runcanon_mine. This may take several minutes. Wait for the result.
8	runcanon_export	Use runcanon_export with harnesses ["claude"] and projectPath set to the absolute path to this repo.
9	runcanon_get_goals	Use runcanon_get_goals and list workspace goals.
10	runcanon_set_goals	Use runcanon_set_goals with goals ["Automate CVE triage", "Audit dependencies for known CVEs"].
11	runcanon_get_stats	Use runcanon_get_stats and report goal alignment and skill counts.
12	runcanon_sync_skills	Use runcanon_sync_skills with harnesses ["claude"] and projectPath set to the absolute path to this repo.
13	runcanon_list_assignments	Use runcanon_list_assignments and list org assignments for my user.
14	runcanon_import_skills_from_git	Use runcanon_import_skills_from_git with repoUrl "https://github.com/anthropics/skills", branch "main", destination "proposal", enrich false. (May 403 for engineer; retry as admin narrative.)
15	runcanon_emit_event	Use runcanon_emit_event with sessionId "demo-session-1", actor "agent", type "tool_call", action "search_codebase", intent "CVE triage", outcome "success".

After sync or export, verify on host:

```
ls .claude/skills/*/SKILL.md
```

Invoke the skill in Claude Code:

```
/audit-dependencies-known-cves
```

Or:

```
Audit core package dependencies for known CVEs in this project
```

Demo line: "Same skill spec, native Claude Code format, invoked as a first-class skill."

Phase 8. Admin: org library, validation, sharing (12 min)

Assignments use org library skills only. Workspace skills must be published first.

8.1 Publish workspace skill to org library

- 1. Sign in as admin
- 2. Settings, switch workspace to Blessed Uyo's workspace (to publish engineer's skill)
- 3. Admin, Org library
- 4. Under Unpublished workspace skills, Publish on audit-dependencies-known-cves

Or publish from admin's own workspace skills if present.

8.2 Git import with validation (browser)

Admin, Org library, Import from Git

Field	Example
Repo URL	https://github.com/your-org/uc-security-skills
Branch	main
Token	(private repos only; not stored)
Destination	org
LLM enrich	On

Review import assessment scores in the UI.

8.3 Create and edit org skill

Admin, Org library, Create skill

Author markdown in browser, save, appears in org library with version history on edit.

8.4 Promotion queue (engineer to org)

When a non-admin publishes, skill enters Admin, Promotions for curator approval before org library.

1. As engineer: attempt publish (if UI exposes it)
2. As admin: Promotions, Approve

8.5 Assign skill to user

1. Admin, Assignments
2. Org skill dropdown now lists published skills
3. Target type: User
4. Target: blessed@runcanon.ai
5. Optional: Mandatory, expiry, project scope
6. Assign skill

8.6 Admin metrics and audit

Admin, Metrics

- Published skills, assignments, sync/export counts
- Signed bundle export (if configured)

Admin, Promotions, Groups, Users as needed for enterprise story.

Phase 9. Engineer pulls shared org skills (8 min)

Sign in as blessed again. CLI and MCP must use engineer credentials:

```
runcanon logout
runcanon login --server http://127.0.0.1:3000 --email blessed@runcanon.ai --password
'YOUR_ENGINEER_PASSWORD'
```

9.1 MCP: list assignments and sync

In Claude Code:

```
Use runcanon_list_assignments and describe mandatory org skills assigned to me.
Use runcanon_sync_skills with harnesses ["claude"] and projectPath set to the absolute path to
this repo.
```

9.2 CLI: export entitled skills

```
runcanon export -h claude --project .
ls .claude/skills/*/SKILL.md
```

9.3 Use skill in Claude Code

```
Audit dashboard dependencies for known CVEs
```

Show Claude following workflow steps from synced SKILL.md.

Demo line: "Admin assigns once. Engineer syncs. LLM uses harness-native skills locally."

Phase 10. Architecture and role check (5 min)

10.1 Guide to Packages

Package	Role in your story
@runcanon/core	Clusters trajectories, scores outcomes, goal alignment
@runcanon/spec	Canonical skill schema
@runcanon/cli	init, mine, review, export, import, login
@runcanon/mcp	12 connected tools for Claude Code
@runcanon/dashboard	Proposals, skills, trajectories, org admin
@runcanon/platform	Workspaces, org library, assignments, metrics
@runcanon/harness-*	Claude, Cursor, Copilot, Codex, Antigravity export

10.2 Role boundaries

As `blessed@runcanon.ai`:

- Providers and Users blocked
- Can mine, approve in own workspace (Proposals), export, MCP sync
- Cannot assign org skills or configure LLM keys

As `admin@runcanon.ai`:

- Full org admin, providers, promotions, assignments, metrics

End-to-end story (one slide)

Trajectory JSONL (real agent sessions, tool order preserved) | mine (CLI or dashboard or MCP) Clusters + LLM proposals | human approval (Proposals) Active skill library (metrics, goal alignment) | publish (org library) + assign (admin) Entitled skills for engineers | export / sync_skills / get_skill writeLocally Harness files (.claude, .cursor, .github, ...) | Claude Code / Cursor / Copilot Repeatable, governed agent workflows | new trajectories + re-mine Update proposals (living library, still human-gated)

Full demo checklist

[] 0	Dashboard up at <code>http://127.0.0.1:3000</code> , CLI installed from Guide
[] 1	Admin: LLM provider tested
[] 1	Admin: user created (or blessed confirmed)
[] 1	Admin: group created (optional)
[] 1	Admin: CLI login + <code>whoami</code>
[] 2	Engineer: browser login + harnesses saved
[] 2	Engineer: CLI login + <code>whoami</code>
[] 3	<code>init --project .</code>
[] 3	Trajectory JSONL seeded
[] 3	mine (all <code>--source</code> paths)
[] 3	Trajectories + Proposals in dashboard
[] 4	Approve at least 1 proposal; reject at least 1 optional
[] 4	Active skill with metrics
[] 4	Goals set via <code>runcanon</code> goals; Goal Alignment on Overview
[] 3	<code>export -h all --project .</code>
[] 3	import (admin, optional)
[] 5	Re-mine or upload, update proposal story
[] 5	Autonomy default = review
[] 6	Claude MCP wired (<code>claude mcp add runcanon -s project -- runcanon mcp</code>)
[] 7	All 14 MCP tools demonstrated
[] 7	Skill invoked in Claude Code (<code>/skill</code> or <code>trigger</code>)
[] 8	Org publish + assignment

[] 9	Engineer sync + local .claude/skills verified
[] 10	Architecture + role check

45-minute Show and Tell run order

Min	Segment
2	Paper contrast: ordered trajectories + human review vs static prompts
3	Conceptual layers diagram; confirm dashboard + CLI ready
5	Phase 1: Admin providers, user, group
5	Phase 2 to 3: Engineer CLI login, init, seed JSONL, mine
5	Phase 4: Proposals approve, Skills metrics, goal alignment
5	Phase 6 to 7: Claude MCP setup, 6 MCP tools live (list, get, sync, mine, stats, emit)
5	Phase 8: Org publish + assign
5	Phase 9: Engineer sync, invoke skill in Claude Code
5	Phase 5 + 10: Living updates, autonomy, architecture
5	Q&A

Troubleshooting

Issue	Fix
Empty Skills page	Hard refresh; ensure container rebuilt after registry fix; do not set RUNCANON_PROJECT_PATH on dashboard container
Empty Assignments dropdown	Publish skill in Admin to Org library first (orgSkills was empty)
No proposals	Admin to Providers enabled + tested
Mining hangs 2 to 5 min	Normal; keep tab open
sync_skills pruned local files	Pass the full Mac path for projectPath, not a placeholder string
MCP tools missing	Restart Claude Code; claude mcp list; runcanon whoami
approve_proposal blocked in MCP	Expected in connected mode; use dashboard
Wrong workspace	runcanon whoami; switch workspace in Settings
Stale UI after platform update	Hard refresh browser
Duplicate proposed skills	Reject duplicates in Proposals; approve the strongest candidate

Quick reference, every CLI command

Install CLI from Guide first, then:

```
# Auth
runcanon login --server http://127.0.0.1:3000 --email USER --password 'PASS'
runcanon login --server http://127.0.0.1:3000 --browser
runcanon whoami
runcanon logout

# Project lifecycle
runcanon init --project .
runcanon mine --project . --source .runcanon/trajectories --source PATH ...
runcanon review --project .
runcanon export -h all --project .
runcanon export -h claude --project .
runcanon export -h all --project . --workspace

# Goals
runcanon goals list
runcanon goals set "Goal one" "Goal two"
runcanon goals add "Another goal"
runcanon goals clear
runcanon goals set "Goal" --workspace WORKSPACE_ID # admin only

# Git import (curator+)
runcanon import --repo URL --branch main --destination org

# Meta
runcanon --version
runcanon --help
```

Quick reference, Claude Code MCP setup

```
claude mcp add runcanon -s project -- runcanon mcp
```


Quick reference, Claude Code MCP prompts

```
Use runcanon_list_skills.  
Use runcanon_get_skill for SKILL_ID with writeLocally true harnesses ["claude"] and  
projectPath set to the absolute path to this repo.  
Use runcanon_list_proposals.  
Use runcanon_mine.  
Use runcanon_export with harnesses ["claude"].  
Use runcanon_get_goals.  
Use runcanon_set_goals with goals ["Automate CVE triage", "Audit dependencies"].  
Use runcanon_get_stats.  
Use runcanon_sync_skills with harnesses ["claude"] and projectPath set to the absolute path to  
this repo.  
Use runcanon_list_assignments.  
Use runcanon_import_skills_from_git with repoUrl ".." destination "org".  
Use runcanon_emit_event with sessionId "demo-1" actor "agent" type "tool_call" action "Read"  
outcome "success".
```